COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	7/31/2026	**DATE SUBMITTED:**	
NAME:	Jerry Matthew L. Larracas	**INSTRUCTOR:**	Engr. Jimlord Quejado

SEATWORK NO. 4.1 STACKS

1. PROCEDURE OUTPUT

ILOA

Source code	Output

`//Push`
`stack1.push('J');`
`std::cout << "is the stack still empty?" << stack1.empty() << std::endl;`
`std::cout << "\nCurrent top of the stack 1: "<< stack1.top() << std::endl;`
 	
`stack1.push('e');`
`std::cout << "Current top of the stack 1: "<< stack1.top() << std::endl;`
 	
`stack1.push('r');`
`std::cout << "Current top of the stack 1: "<< stack1.top() << std::endl;`
 	
`stack1.push('r');`
`std::cout << "Current top of the stack 1: "<< stack1.top() << std::endl;`
 	
`stack1.push('y');`
`std::cout << "Current top of the stack 1: "<< stack1.top() << std::endl;`
	`Current top of the stack 1: J`
	`Current top of the stack 1: e`
	`Current top of the stack 1: r`
	`Current top of the stack 1: r`
	`Current top of the stack 1: y`

`//isEmpty` `std::cout << "\nis the stack empty?" << stack1.empty() << std::endl;`	`is the stack empty?1`

`//swap`
`//declare a new stack variable`
`std::stack<char> stack2;`
 	
`stack2.push('i');`
`stack1.swap(stack2);`
`std::cout << "\nSwapping the new stack 2 and stack 1";`
`std::cout << "\nCurrent top of the stack 1: "<< stack1.top() << std::endl;`
`std::cout << "Current top of the stack 2: "<< stack2.top() << std::endl;`
	`Swapping the new stack 2 and stack 1`
	`Current top of the stack 1: i`
	`Current top of the stack 2: y`

`//emplace`
`stack1.emplace('S');`
`std::cout << "\nEmplacing new data at top..\n";`
`std::cout << "Current top of the stack 1: " << stack1.top() << std::endl;`
	`Emplacing new data at top..`
	`Current top of the stack 1: S`

`//Pop`
`std::cout <<"\npopping the current top of the stack 1..";`
`stack1.pop();`
`std::cout << "\nShow the current top of the stack 1: " << stack1.top();`
	`popping the current top of the stack 1..`
	`Show the current top of the stack 1: i`

Observation: This activity shows the datas that are being implement or pushed are getting stacked, which means it creates a stack of data from the top. Stacks also checks if it is empty and it can also swaps data, emplaceing new data or adding a new data at the top and also deleting of popping the current top stack.

ILOB

Source Code	Output

```
#include <iostream>

//Global variables
//static
const int maxSize = 10;
char stackArr[maxSize];
char top = -1, newData;

//Function prototypes
// Top, Push, Pop, isEmpty, isFull
```

```
void Push(){
  //error catching
  if(isFull()){
  std::cout << "Stack Overflow!" << std::endl;
  return;
  }

  //ask the user what they want to insert
  std::cout << "What character do you want to push? ";
  std::cin >> newData;

  //storing to the stack
  stackArr[++top] = newData;
}
```

```
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
1
What character do you want to push? J


====================================
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
1
What character do you want to push? E


====================================
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
1
What character do you want to push? R


====================================
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
1
What character do you want to push? R


====================================
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
1
What character do you want to push? Y
```

```
void Top(){
// error catching:
  if(isEmpty())
  return;
//prints the top element
  std::cout << "The top element is: "<< stackArr[top] << std::endl;
}
```

```
====================================
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
5
The top element is: Y
```

```
void Pop(){
  // error catching
  if(isEmpty()){
    std::cout << "Stack Underflow!" << std::endl;
    return;
  }

  //display the item to be popped
  std::cout << "Popping: " << stackArr[top] << std::endl;
  //we pop the element from the stack
  top--;
}
```

```
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
2
Popping:


Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
6
```

```
void Top(){
// error catching:
  if(isEmpty())
  return;
//prints the top element
  std::cout << "The top element is: "<< stackArr[top] << std::endl;
}

void display(){
  while(stackArr[top] != isEmpty()){
    std::cout << stackArr[top] << " ";
    top--;
  }
  return;
}
```

```
Stack operations:
[1] push [2] pop [3] isEmpty [4] isFull [5] Top [6] Display
3
Is the stack empty?0
```

Observation:

This activity is also the same as the ILO Activity but this time it is declared in function which a user can interact with the stack operations such as push, pop, isEmpty, isFull, displaying the current top and displaying all the stacks. The user will select which option the user wants, and if it is push, the user will get asked what character the user wants to push, if pop, the user will able to pop or delete the current character that is stored currently at the top and isEmpty option if the user wants to check if the stack is empty. The top option is simply displays the current top character that was pushed by the user and lastly the display option, this simply displays all of the characters that the user pushed or stored in the stack.

ILOB 2

```cpp
#ifndef STACK_LIST_H
#define STACK_LIST_H

//class node that contains data and the next:
template <typename T>
class Node{
  public:
  T data;
  Node* next;
};

//Global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

// Operations
//PUSH
template<typename T>
void push(T newData){
  //create a new node and assight the newData
  Node<T>* newNode = new Node<T>;
  newNode->data = newData;

  //new node points to current top then becames the new top
  newNode->next = head<T>;
  head<T> = newNode;
}

//POP
template<typename T>
T pop(){
  //check if the stack is empty
  if(head<T> == nullptr){
    std::cout << "stack Underflow! \n";
  }
  //save the current top node
  Node<T>* temp = head<T>;
  //store its data
  T tempVal = temp->data;

  // move the head to the next node
  head<T> = head<T>->next;
  // free the removed node
  delete temp;
  //return the removed value
  return tempVal;
}
```

```cpp
#include<iostream>
#include "stackList.h"

int main(){
  std::cout << "=== PUSHING 'j' 'e' 'r' ONTO THE STACK === \n ";
  //push
  push('j');
  push('e');
  push('r');

  //display the current top
  Top<char>();
  displayALL<char>();
  std::cout<<"\n ------ Popping all the elements ------ \n";

  while(!isEmpty<char>()){
    std::cout <<"Popped: " << pop<char>() << std::endl;
  }

  std::cout<<"\n ------ Testing the Underflow ------ \n";
  pop<char>();

  return 0;
}
```

```cpp
// isEmpty
template<typename T>
bool isEmpty(){
  return head<T> == nullptr;
}

//top
template<typename T>
void Top(){
  //error catching
  if(isEmpty<T>()){
    std::cout<<"Stack Underflow! \n";
    return;
  }
  //display the Top node
  std::cout <<"Top of stack: " << head<T>->data << std::endl;
}

//display All
template<typename T>
void displayALL(){
  if(head<T> == nullptr){
    std::cout << "stack underflow!\n";
    return;
  }
  std::cout << "Displaying the stack: \n";
  Node<T>* temp = head<T>;
  while(temp != nullptr){
    std::cout << temp->data << "\n";
    temp = temp->next;
  }
  std::cout << "\n";
}

#endif
```

```
=== PUSHING 'j' 'e' 'r' ONTO THE STACK ===
 Top of stack: r
Displaying the stack:
r
e
j


 ------ Popping all the elements ------
Popped: r
Popped: e
Popped: j

 ------ Testing the Underflow ------
stack Underflow!
```

Observation/Code Analysis

This ILOB part 2 is stackList, it is a combination of stacks and linked list where we have to create a node and transfer data from one to another and stacking them. The system will push the three elements ‘j’ ’e’ ’r’ onto the stack with a syntax like “push(‘j’);” to push an element. Then the top of the stack should be the last element that the system pushed ‘r’ and to do this, the head node should point to data to display the top of the stack. When displaying the stack, we create a displayALL function where the node points to temporary variable equals to head, then while loop with a condition of temp is not equal to nullptr then it will display temporary datas that were stacked from top to bottom. THe system will then pop all the elements or deleting them and the system will test if the data still exist by testing the underflow, this function is a simply if else such as if head is equal to nullptr, the system will display stack is underflow meaning the stack is already empty or any data that was stacked recently were deleted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity

ILO C: Solve problems using an implementation of stack:
The following problem definition and algorithm is provided for checking the balancing of symbols. Create an implementation using stacks. Your output must include the following:
a. Stack using Arrays
b. Stack using Linked Lists
c. (Optional) Stack using C++ STL
Problem Definition:
Stacks can be used to check whether the given expression has balanced symbols. This algorithm is very useful in compilers. Each time the parser reads one character at a time. If the character is an opening delimiter such as (, {, or [- then it is written to the stack. When a closing delimiter is encountered like), }, or]-the stack is popped. The opening and closing delimiters are then compared. If they match, the parsing of the string continues. If they do not match, the parser indicates that there is an error on the line.
Steps:
1. Create a Stack.
2. While(end of input is not reached) {
a) If the character read is not a symbol to be balanced, ignore it.
b) If the character is an opening symbol, push it onto the stack.
c) If it is a closing symbol:
i) Report an error if the stack is empty.
ii) Otherwise, pop the stack.
d) If the symbol popped is not the corresponding opening symbol, report an error.
3. At the end of input, if the stack is not empty: report an error.

Self-Checking:
For the following cases, complete the table using the code you created.

Expression	Valid? (Y/N)	Output(Console Screenshot)	Analysis
(A+B)+(C-D)			
((A+B)+(C-D)			
(A+B)+[C-D])			
((A+B]+[C-D]}			

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.